



Curso: BACKEND

Tema: FUNDAMENTOS DE JAVA

FUNDAMENTOS DE JAVA

¿QUÉ ES UNA VARIABLE?

Una variable es un espacio en memoria que almacena un valor.

💡 Una variable tiene:

- Tipo de dato
- Nombre
- Valor

```
public static void main(String[] args) {  
    int edad = 25;  
}
```



TIPOS DE DATOS EN JAVA

TIPOS DE DATOS EN JAVA

En Java, los datos que utiliza un programa deben tener un tipo definido. Esto permite que el lenguaje controle cómo se almacenan y manipulan los valores en memoria. Java es un lenguaje de tipado fuerte, lo que significa que no permite asignar valores incompatibles a una variable.

Clasificación:

Los tipos de datos en Java se dividen en:

- ◆ Tipos primitivos → almacenan valores directamente
- ◆ Tipos no primitivos → almacenan referencias a objetos

TIPOS PRIMITIVOS

Los tipos primitivos son los más básicos en Java. Se utilizan para almacenar valores simples y ocupan una cantidad fija de memoria.

Características:

- No son objetos
- Se almacenan directamente en memoria
- Son más rápidos que los objetos

Tipo	Ejemplo	Uso
int	10	Números enteros
double	10.5	Números decimales
char	'A'	Caracteres
boolean	true	Valores lógicos

EJEMPLO TIPOS PRIMITIVOS

Para usar una variable en Java, se debe indicar su tipo antes del nombre.

- **int** : almacena números enteros
- **double** : permite valores con decimales
- **char** : almacena un solo carácter
- **boolean** : solo puede ser true o false

```
int edad = 30;  
double sueldo = 2500.75;  
char inicial = 'P';  
boolean activo = true;
```

TIPO STRING (NO PRIMITIVO)

String es un tipo de dato no primitivo que se utiliza para almacenar texto. A diferencia de los primitivos, los **String** son objetos.

Características:

- Se escriben entre comillas dobles
- Permiten almacenar palabras o frases completas
- Internamente son objetos de la clase **String**

```
public static void main(String[] args) {  
    String nombre = "Paul";  
}
```

***Importante:** Aunque parece simple, **String** es uno de los tipos más utilizados en aplicaciones reales.*

SALIDA POR CONSOLA

Java permite mostrar información en pantalla mediante la consola utilizando la clase System.

Detalle de lo ocurrido:

- System: clase del sistema
- Out: salida estándar
- Println: imprime y hace salto de línea

```
public static void main(String[] args) {  
    String nombre = "Paul";  
    System.out.println(nombre);  
}
```

INTRODUCCIÓN A SCANNER

Scanner es una clase de Java que permite capturar datos ingresados por el usuario desde el teclado. Se utiliza cuando queremos que el programa sea interactivo.

¿Por qué es importante?

Sin entrada de datos, los programas solo muestran información. Con Scanner, los programas pueden reaccionar a lo que el usuario ingresa

```
import java.util.Scanner;  
  
public class Main
```



CREACIÓN DEL SCANNER

Para utilizar Scanner, debemos crear un objeto que lea los datos desde la entrada estándar (teclado).

Detalle:

Scanner : tipo de dato

scanner : nombre de la variable

System.in : entrada desde teclado

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
}
```

LECTURA DE DATOS

Scanner tiene distintos métodos dependiendo del tipo de dato:

Detalle:

- `nextLine()` : texto completo
- `nextInt()` : número entero
- `nextDouble()` : número decimal
- `nextBoolean()` : valor lógico

```
public static void main(String[] args) {  
  
    Scanner scanner = new Scanner(System.in);  
  
    String nombre = scanner.nextLine();  
    int edad = scanner.nextInt();  
}
```

EJEMPLO COMPLETO

El programa solicita un dato, lo guarda en una variable y luego lo muestra.

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    System.out.print("Ingresa tu nombre: ");  
    String nombre = scanner.nextLine();  
    System.out.println("Hola " + nombre);  
    scanner.close();  
}
```

BUENAS PRÁCTICAS

- Usar nombres descriptivos para variables
- Cerrar el Scanner (scanner.close())
- Mantener el código ordenado
- Validar entradas del usuario

```
10 public class Main
11 {
12     public static void main(String[] args) {
13
14         Scanner scanner = new Scanner(System.in);
15
16         System.out.print("Ingresa tu nombre: ");
17         String nombre = scanner.nextLine();
18
19         System.out.println("Hola " + nombre);
20
21         scanner.close();
22     }
23 }
```

input

```
Ingresa tu nombre: Nike
Hola Nike
```

CONDICIONALES

¿QUÉ ES UNA CONDICIÓN?

Una condición es una expresión que devuelve un valor:

👉 true (verdadero)

👉 false (falso)

Importante:

Las condiciones se utilizan para tomar decisiones dentro de un programa.

```
public static void main(String[] args) {  
    int edad = 18;  
    boolean esMayor = edad >= 18;  
}
```


¿QUÉ ES UN IF?

La estructura **if** permite ejecutar un bloque de código solo si una condición es verdadera.

Interpretación:

Si la condición es verdadera → se ejecuta

Si es falsa → se ignora

```
if (condicion) {  
    // código a ejecutar  
}
```

EJEMPLO BASICO

La estructura **if** permite ejecutar un bloque de código solo si una condición es verdadera.

Interpretación:

Si la condición es verdadera → se ejecuta

Si es falsa → se ignora

```
8
9 public class Main
10 {
11     public static void main(String[] args) {
12         int edad = 20;
13
14         if (edad >= 18) {
15             System.out.println("Eres mayor de edad");
16         }
17     }
18 }
19
20 }
```

input

Eres mayor de edad

OPERADORES RELACIONALES

<i>Operador</i>	<i>Significado</i>
==	Igual
!=	Diferente
>	Mayor
<	Menor
>=	Mayor o igual
<=	Menor o igual

IF / ELSE

Permite definir un camino alternativo cuando la condición no se cumple.

Interpretación:

Si la condición es verdadera → se ejecuta

Si es falsa → ingresa al else

```
public static void main(String[] args) {  
    if (condicion) {  
        // verdadero  
    } else {  
        // falso  
    }  
}
```

EJEMPLO IF / ELSE

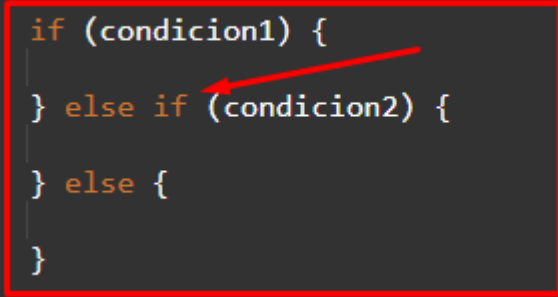
```
11 public static void main(String[] args) {  
12  
13     int edad = 16;  
14  
15     if (edad >= 18) {  
16         System.out.println("Mayor de edad");  
17     } else {  
18         System.out.println("Menor de edad");  
19     }  
20 }  
21 }
```

Menor de edad

IF / ELSE IF

Permite evaluar múltiples condiciones en orden.

```
public static void main(String[] args) {  
    if (condicion1) {  
    } else if (condicion2) {  
    } else {  
    }  
}
```



EJEMPLO IF / ELSE IF

```
11 public static void main(String[] args) {  
12  
13     int nota = 15;  
14  
15     if (nota >= 18) {  
16         System.out.println("Excelente");  
17     } else if (nota >= 13) {  
18         System.out.println("Aprobado");  
19     } else {  
20         System.out.println("Desaprobado");  
21     }  
22 }  
23 }
```

Aprobado ←

OPERADORES LÓGICOS

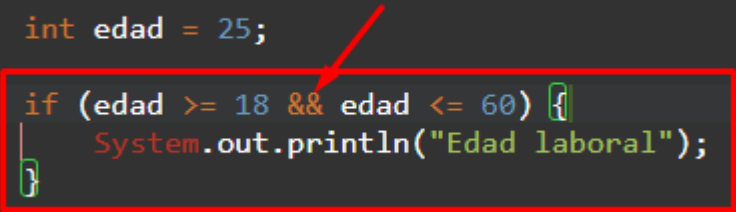
Permiten combinar condiciones

Operador	Significado
&&	AND (y)
!	NOT (negación)
	o

EJEMPLO OPERADORES LÓGICOS

Permiten combinar condiciones

```
public static void main(String[] args) {  
    int edad = 25;  
    if (edad >= 18 && edad <= 60) {  
        System.out.println("Edad laboral");  
    }  
}
```



EJERCICIO EN CLASE

Crear un programa que:

❑ Pida un número

Determine en consola si es:

- Positivo
- Negativo
- Cero



EJERCICIO EN CLASE

Crear un programa que:

Pida una nota (0–20)

Muestre:

- Excelente
- Aprobado
- Desaprobado



EJERCICIO EN CLASE

Crear un programa que:

Pida edad

Determine:

- Niño (0–12)
- Adolescente (13–17)
- Adulto (18–59)
- Adulto mayor (60+)



Las decisiones en programación se construyen con condiciones.



¿QUÉ ES SWITCH?

La estructura switch permite ejecutar diferentes bloques de código según el valor de una variable. Se utiliza cuando existen múltiples opciones posibles basadas en un mismo valor. Es una alternativa más ordenada al uso de múltiples if-else

```
public static void main(String[] args) {  
  
    switch (variable) {  
        case valor1:  
            // código  
            break;  
        case valor2:  
            // código  
            break;  
        default:  
            // código por defecto  
    }  
}
```

¿QUÉ ES SWITCH?

La estructura switch permite ejecutar diferentes bloques de código según el valor de una variable. Se utiliza cuando existen múltiples opciones posibles basadas en un mismo valor. Es una alternativa más ordenada al uso de múltiples if-else

switch → evalúa una variable

case → posibles valores

break → detiene la ejecución

default → opción por defecto

```
public static void main(String[] args) {  
    switch (variable) {  
        case valor1:  
            // código  
            break;  
        case valor2:  
            // código  
            break;  
        default:  
            // código por defecto  
    }  
}
```

¿QUÉ PASA SIN BREAK?

Se ejecuta todo los print

```
11 public static void main(String[] args) {  
12  
13     int dia = 3;  
14  
15     switch (dia) {  
16         case 1:  
17             System.out.println("Lunes");  
18             break;  
19         case 2:  
20             System.out.println("Martes");  
21             break;  
22         case 3:  
23             System.out.println("Miércoles");  
24             break;  
25         default:  
26             System.out.println("Día inválido");  
27     }  
28 }  
29 }
```

Miércoles

¿QUÉ SON LAS ESTRUCTURAS REPETITIVAS?

Las estructuras repetitivas (o bucles) permiten ejecutar un bloque de código varias veces.

Se usan cuando:

- Necesitamos repetir una acción
- Queremos recorrer datos
- No sabemos cuántas veces se ejecutará el código

TIPOS DE BUCLES EN JAVA

En Java existen 3 principales estructuras repetitivas:

- **For**
- **While**
- **do-while**

Cada una tiene un uso específico dependiendo del escenario.

BUCLE FOR

El bucle for se usa cuando sabemos cuántas veces queremos repetir una acción.

- For
- While
- do-while

Cada una tiene un uso específico dependiendo del escenario.

```
public static void main(String[] args) {  
    for(inicialización; condición; incremento) {  
        // código a ejecutar  
    }  
}
```

BUCLE FOR

Imprime números del 1 al 5

Explicación:

int i = 1; // *inicia en 1*

i <= 5; // *condición*

i++; // *incremento*

- Se ejecuta mientras la condición sea verdadera.
- En cada vuelta, el valor cambia.

```
8
9 public class Main
10 {
11     public static void main(String[] args) {
12
13         for(int i = 1; i <= 5; i++) {
14             System.out.println("Número: " + i);
15         }
16     }
17 }
```

```
Número: 1
Número: 2
Número: 3
Número: 4
Número: 5
```

BUCLE WHILE

El bucle while se usa cuando no sabemos cuántas veces se repetirá el código.

```
public static void main(String[] args) {  
    while(condición) {  
        // código  
    }  
}
```

BUCLE WHILE

!!!**Importante!** :Si no actualizas la variable se convierte en bucle infinito

```
9 public class Main
10 {
11     public static void main(String[] args) {
12
13         int i = 1;
14
15         while(i <= 5) {
16             System.out.println("Número: " + i);
17             i++;
18         }
19     }
20 }
```

```
Número: 1
Número: 2
Número: 3
Número: 4
Número: 5
```

BUCLE DO-WHILE

El do-while ejecuta el código al menos una vez, incluso si la condición es falsa.

```
public static void main(String[] args) {  
    do {  
        // código  
    } while(condición);  
}
```

BUCLE DO-WHILE

Importante!! Primero ejecuta, luego evalúa

```
10 {  
11     public static void main(String[] args) {  
12  
13         int i = 1;  
14  
15         do {  
16             System.out.println("Número: " + i);  
17             i++;  
18         } while(i <= 5);  
19     }  
20 }
```

```
Número: 1  
Número: 2  
Número: 3  
Número: 4  
Número: 5
```

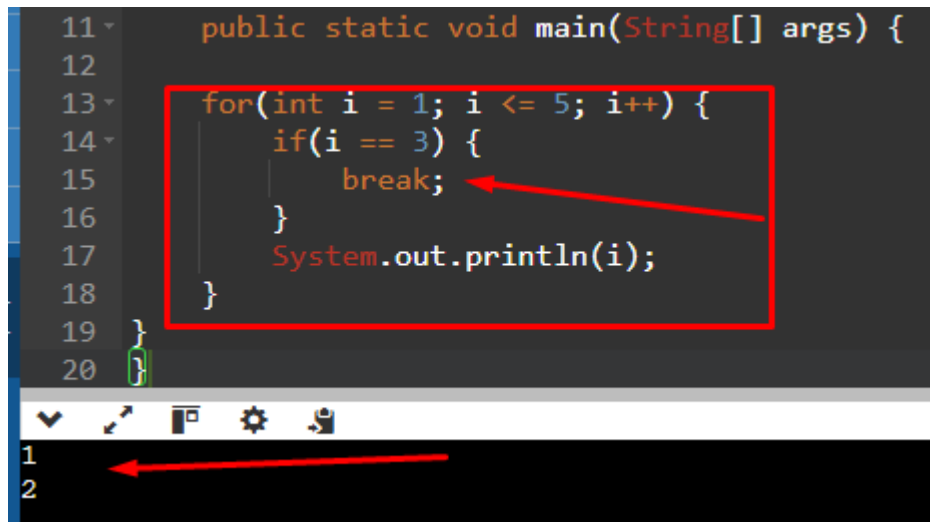

DIFERENCIAS

<i>Bucle</i>	<i>¿Cuándo usarlo?</i>
for	Sabes cuántas veces repetir
while	No sabes cuántas veces
do-while	Debe ejecutarse mínimo una vez

BREAK Y CONTINUE

- **break** → rompe el bucle
- **continue** → salta a la siguiente iteración

```
11 public static void main(String[] args) {  
12  
13     for(int i = 1; i <= 5; i++) {  
14         if(i == 3) {  
15             break;  
16         }  
17         System.out.println(i);  
18     }  
19 }  
20 }
```

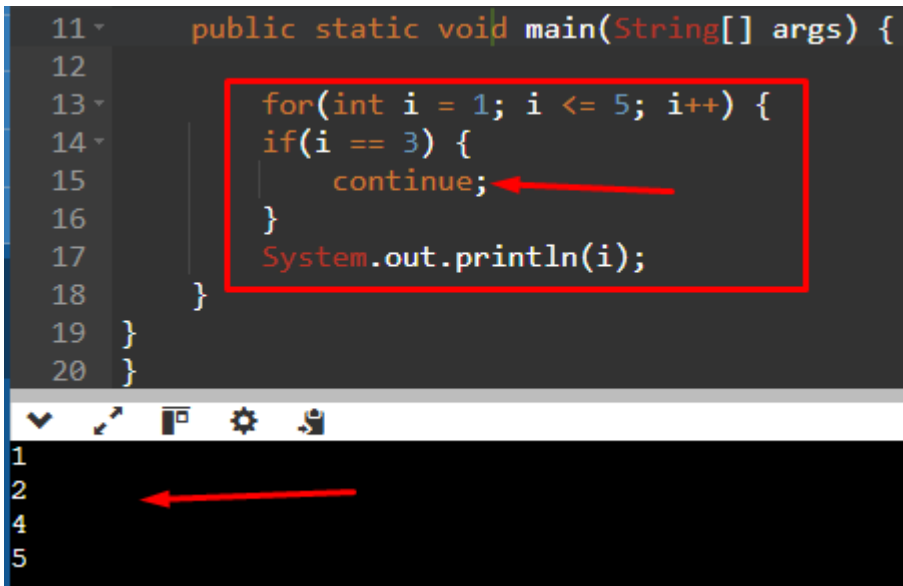


1
2

BREAK Y CONTINUE

- **break** → rompe el bucle
- **continue** → salta a la siguiente iteración

```
11 public static void main(String[] args) {  
12  
13     for(int i = 1; i <= 5; i++) {  
14         if(i == 3) {  
15             continue;  
16         }  
17         System.out.println(i);  
18     }  
19 }  
20 }
```



BUCLES ANIDADOS

Un bucle dentro de otro bucle.

```
11 public static void main(String[] args) {  
12  
13     for(int i = 1; i <= 3; i++) {  
14         for(int j = 1; j <= 2; j++) {  
15             System.out.println("i=" + i + " j=" + j);  
16         }  
17     }  
18 }  
19 }
```

i=1 j=1
i=1 j=2
i=2 j=1
i=2 j=2
i=3 j=1
i=3 j=2

EJERCICIO EN CLASE

Crear un programa que:

- ✓ Imprima números del 1 al 10
- ✓ Use un bucle for



EJERCICIO EN CLASE

Crear un programa que:

- ✓ Pida un número
- ✓ Muestre su tabla de multiplicar
- ✓ Use while



EJERCICIO EN CLASE

Crear un programa que:

- ✓ Muestre todos los números pares del 1 al 50
- ✓ Use for
- ✓ Use if



RETO

Crear un programa que:
✓ Dibuje un triángulo con *

```
*  
**  
***  
****
```



RESUMEN

- ✓ Los bucles permiten repetir código
- ✓ Existen 3 tipos principales
- ✓ Son clave para lógica de programación
- ✓ Se combinan con condicionales

PROGRAMACIÓN ORIENTADA A OBJETOS (POO)

¿QUÉ ES LA POO?

La Programación Orientada a Objetos (POO) es un paradigma de programación que permite modelar problemas del mundo real usando “objetos”. Un objeto representa una entidad con:

- Atributos (datos o características)
- Comportamientos (acciones o métodos)

Por ejemplo:

Un “Auto” puede tener:

- Atributos: color, marca, velocidad
- Métodos: acelerar(), frenar()

¿POR QUÉ USAR POO?

La POO permite:

- ✓ Organizar mejor el código
- ✓ Reutilizar componentes
- ✓ Facilitar mantenimiento
- ✓ Modelar problemas reales
- ✓ Crear sistemas escalables

En resumen es el estándar en desarrollo profesional moderno.

CONCEPTOS CLAVE DE POO

Los pilares fundamentales de la POO son:

- ✓ Clases
- ✓ Objetos
- ✓ Encapsulamiento
- ✓ Herencia
- ✓ Polimorfismo
- ✓ Abstracción

¿QUÉ ES UNA CLASE?

Una clase es una plantilla o modelo que define cómo será un objeto.
Define:

- ✓ Atributos (variables)
- ✓ Métodos (funciones)

Es como un “molde” para crear objetos.

EJEMPLO DE CLASE EN JAVA

Esta clase define cómo será un Auto.

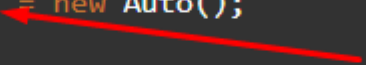
```
public class Auto {  
    String marca;  
    String color;  
    int velocidad;  
  
    public void acelerar() {  
        System.out.println("El auto está acelerando");  
    }  
}
```

¿QUÉ ES UN OBJETO?

Un objeto es una instancia de una clase.

Es decir: Es un elemento real creado a partir de una clase

```
9 public class Main {  
10     public static void main(String[] args) {  
11         Auto miAuto = new Auto();  
12           
13         miAuto.marca = "Toyota";  
14         miAuto.color = "Rojo";  
15           
16         miAuto.acelerar();  
17     }  
18 }
```



ENCAPSULAMIENTO

El encapsulamiento consiste en proteger los datos de un objeto.

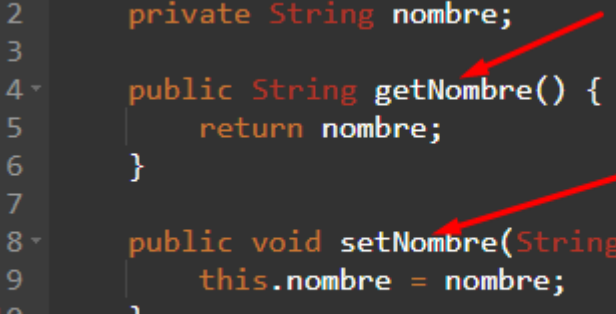
Se logra usando:

- ✓ `private` → restringe acceso
- ✓ `public` → permite acceso controlado

GETTERS Y SETTERS

Permiten acceder a atributos privados de forma segura.

```
1 public class Persona {  
2     private String nombre;  
3  
4     public String getNombre() {  
5         return nombre;  
6     }  
7  
8     public void setNombre(String nombre) {  
9         this.nombre = nombre;  
10    }  
11 }
```

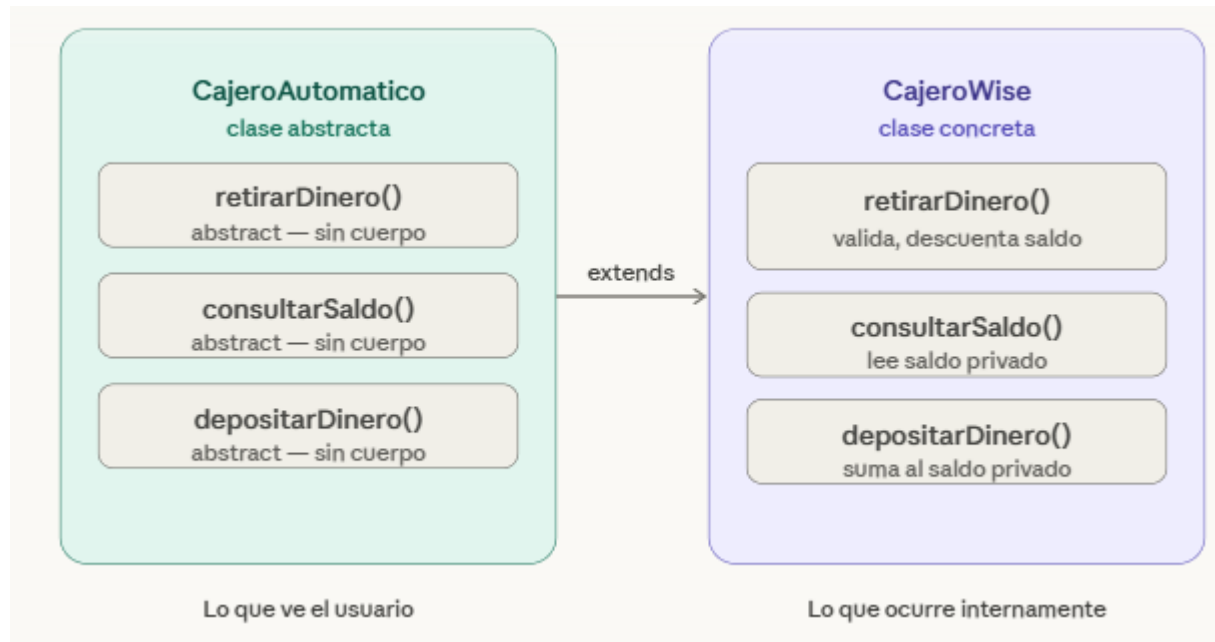


Esto protege los datos y evita errores.

ABSTRACCIÓN

La abstracción consiste en mostrar solo lo necesario y ocultar detalles internos.

Ejemplo: Un usuario usa un cajero automático sin saber cómo funciona internamente.



¿QUÉ ES UNA INTERFACE?

Una interface es un contrato que define qué métodos debe implementar una clase.

No contiene lógica (en lo básico), solo define métodos.

```
1 public interface Vehiculo {  
2     void acelerar();  
3     void frenar();  
4 }
```

IMPLEMENTANDO UNA INTERFACE

La clase está obligada a implementar los métodos.

```
1 public interface Vehiculo {  
2     void acelerar();  
3     void frenar();  
4 }  
5  
6 public class Auto implements Vehiculo {  
7  
8     public void acelerar() {  
9         System.out.println("Auto acelerando");  
10    }  
11  
12    public void frenar() {  
13        System.out.println("Auto frenando");  
14    }  
15 }
```



HERENCIA

La herencia permite que una clase herede atributos y métodos de otra.

Se usa la palabra clave ***extends***.

Nota: Auto hereda de Vehiculo.

```
1 public class Vehiculo {  
2     String marca;  
3 }  
4  
5 public class Auto extends Vehiculo {  
6     int puertas;  
7 }
```



POLIMORFISMO

El polimorfismo permite que un mismo método tenga diferentes comportamientos.

“Muchos comportamientos para una misma acción”.

Importante: El método cambia según el objeto.

```
1 public class Animal {  
2     public void hacerSonido() {  
3         System.out.println("Sonido genérico");  
4     }  
5 }  
6  
7 public class Perro extends Animal {  
8     public void hacerSonido() {  
9         System.out.println("Guau");  
10    }  
11 }
```

EJERCICIO EN CLASE

Crear una clase llamada Persona con:

- ✓ Atributos: nombre, edad
- ✓ Métodos: saludar()

Luego:

- ✓ Crear un objeto
- ✓ Mostrar un mensaje con sus datos

Ejemplo de la Salida:

Hola, mi nombre es Carlos y tengo 20 años.



EJERCICIO EN CLASE

Crear:

- ✓ Interface: **Vehiculo**
- ✓ Clase: Auto
- ✓ Clase: Moto

Cada uno debe implementar:

- ✓ acelerar()
- ✓ frenar()



Conclusiones

